

Dialogues

Implémentation : `dialogue.DialogueDefinitionParser` (validation), `dialogue.session.DialogueSessionEngine` (exécution), `dialogue.render.*` (affichage). Un fichier par dialogue, dans `plugins/RPGQuest/dialogues/*.yaml`.

Comment fonctionne un dialogue

Un dialogue est un graphe de **nœuds** (*nodes*). Chaque nœud a un locuteur (*speaker*), un texte (*text*), et une liste de **choix** (*choices*). L'ouverture d'un dialogue affiche toujours le nœud désigné par *start*.

Un choix cliqué exécute ses *actions* dans l'ordre, puis :

- si les actions contiennent *CLOSE* ou *OPEN_DIALOGUE*, cette transition prend le pas sur tout le reste ;
- sinon, si le choix a un champ *next*, ce nœud (du **même** dialogue) est affiché ensuite ;
- sinon, le dialogue se ferme — jamais de nœud « bloqué » sans issue.

Un même dialogue peut boucler sur lui-même via *next* (menu « hub ») ; c'est un usage normal, pas une erreur.

```
id: rpgquest:guide
start: greeting

nodes:
  greeting:
    speaker: "Guide"
    text: "Bienvenue au village !"
    choices:
      - text: "Très bien, j'y vais."
        actions:
          - type: START_QUEST
            quest: rpgquest:premiers_pas
      - text: "D'accord !"
        actions:
          - type: CLOSE
```

Copier

Comment fonctionnent les choix

Chaque choix peut porter des **conditions** (visibilité) et des **actions** (effet au clic). Un choix sans *conditions* est toujours visible. Les conditions sont

réévaluées **au clic**, pas seulement à l'affichage — un choix affiché puis devenu invalide entre-temps (ex. quête acceptée par un autre biais) est silencieusement ignoré au clic.

Condition	Champs	Effet
QUEST_STATE	quest, state	Vrai si la quête du joueur est dans l'état donné (NOT_STARTED/ACTIVE/READY_TO_TURN_IN/COMPLETED/FAILED/ABANDONED).
HAS_ITEM	material, amount	Vrai si l'inventaire contient au moins <code>amount</code> de <code>material</code> .
HAS_PERMISSION	permission	Vrai si le joueur a cette permission Bukkit.
VARIABLE_EQUALS	key, value	Vrai si la variable joueur <code>key</code> vaut <code>value</code> .

Actions disponibles

Action	Champs	Effet
START_QUEST	quest	Accepte la quête pour le joueur (équivalent <code>/quest accept</code>).
ADVANCE_QUEST	quest	Satisfait immédiatement tous les objectifs de l'étape courante.
TURN_IN_QUEST	quest	Force la remise/complétion de la quête (récompenses accordées).
GIVE_ITEM	material, amount	Donne l'objet au joueur.
TAKE_ITEM	material, amount	Retire l'objet de l'inventaire.
SET_VARIABLE	key, value	Écrit une variable joueur persistante.
RUN_SAFE_COMMAND	command	Exécute une commande console — nom de commande vérifié au chargement contre <code>config.yml</code> → <code>dialogue.allowed-commands</code> .
OPEN_DIALOGUE	dialogue	Ferme le nœud courant et ouvre un autre dialogue.
OPEN_MERCHANT	merchant	Ferme le dialogue et ouvre la vitrine du marchand désigné.
CLOSE	—	Ferme le dialogue immédiatement.

Renderer utilisé

Contrôlé par `config.yml` → `dialogue.renderer`. Deux renderers existent, tous deux dans `dialogue.render` :

Par défaut `paper-dialog`

`PaperDialogRenderer` — vraie fenêtre native Paper avec de vrais boutons cliquables (`io.papermc.paper.dialog.Dialog`). Repose sur une API marquée `@ApiStatus.Experimental` par Paper lui-même (le contrat pourrait changer dans une future version de Paper), mais réelle et fonctionnelle contre la version exacte de Paper ciblée par ce projet.

En cas d'exception à l'exécution, `FallbackDialogueRenderer` bascule automatiquement sur le chat pour ce dialogue — le joueur voit toujours quelque chose.

Alternative `chat`

`ChatDialogueRenderer` — texte `MiniMessage` dans le chat, choix sous forme de lignes numérotées cliquables (`ClickEvent.callback`, une API Adventure stable). Fonctionne avec n'importe quel client, aucune API instable.

À choisir explicitement (`dialogue.renderer: chat`) si vous préférez éviter toute dépendance à une API expérimentale.

Dans les deux cas, `speaker/text`/le libellé des choix sont du `MiniMessage` réellement interprété (couleurs, mise en forme) — jamais affiché sous forme de balises brutes au joueur.

Comment tester un dialogue

1. Placez le fichier dans `plugins/RPGQuest/dialogues/<id>.yaml` (en développement local : `run/plugins/RPGQuest/dialogues/`).
2. Redémarrez le serveur complètement (`stop` puis relancer) — les dialogues ne se rechargent **jamais** à chaud, aucune commande ne l'expose.
3. Deux façons de l'ouvrir en jeu :

```
# À distance, sans PNJ (admin, test)
/dialogue open <joueur> <dialogueId>
```

Copier

```
# Via un PNJ identifié (voir la page PNJ) – clic droit dessus, si
# dialogue dont l'id (sans namespace) correspond exactement à l'i
```